

GSOI 2026 夏季训练赛

Day12 题解

Ehundategh & tfbz

Gioush 大队

2026 年 8 月

目录

T1 抢修计划 (repair)

T2 风眼定位 (locate)

T3 环锁解除 (circle)

T4 风暴止息 (calm)

吐槽时间

Contents

T1 抢修计划 (repair)

T2 风眼定位 (locate)

T3 环锁解除 (circle)

T4 风暴止息 (calm)

吐槽时间

数据点 1 ~ 5 25 Pts

- ▶ 当 $n, k \leq 10$ 时，可以把一次抢修看成搜索树上的一层。状态记录每个区域已经抢修的次数，枚举当前已经开放的区域作为下一次选择。
- ▶ 同一状态只保留已经得到的最大保护价值，搜索深度不超过 k 。这一做法能够完整枚举全部合法行动序列，适合验证后续结论。

数据点 6 ~ 10 50 Pts

- ▶ 假设最后开放到区域 i 。区域 $1 \sim i$ 都必须恰好完成一次首次抢修，因此已经消耗 i 次行动，得到 $\sum_{j=1}^i a_j$ 。
- ▶ 剩余 $k - i$ 次行动不会再影响开放范围。每次都选择 b_j 最大的已开放区域一定最优，额外价值为 $(k - i) \max_{1 \leq j \leq i} b_j$ 。
- ▶ 枚举 i 后直接计算候选答案，时间复杂度为 $\mathcal{O}(n^2)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

数据点 11 ~ 15 75 Pts

- ▶ 特殊性质保证 b 单调不降。最后开放到区域 i 时，已经开放区域中的最大重复抢修价值就是 b_i 。
- ▶ 因此候选答案可以直接写为

$$\sum_{j=1}^i a_j + (k - i)b_i.$$

- ▶ 顺序维护 a 的前缀和即可在线性时间内处理这一档。将该做法与前两档的平方算法合并，可以获得 75 分。

正解 100 Pts

- ▶ 令 $A_i = \sum_{j=1}^i a_j$, $B_i = \max_{1 \leq j \leq i} b_j$ 。最后开放到区域 i 时，最优价值为

$$A_i + (k - i)B_i.$$

- ▶ 必须满足 $1 \leq i \leq \min(n, k)$ 。顺序扫描时同时维护 A_i 与 B_i ，取全部候选的最大值即可。
- ▶ 必要性来自开放区域 i 必须依次完成前 i 次首次抢修。充分性来自完成这 i 次后，把剩余行动全部放在取得 B_i 的区域上。
- ▶ 时间复杂度为 $\mathcal{O}(n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

```
1 long long a[MAXN],b[MAXN];
2
3 void Solve(){
4     scanf("%d%d",&n,&k);
5     for(int i=1;i<=n;i++) scanf("%lld",&a[i]);
6     for(int i=1;i<=n;i++) scanf("%lld",&b[i]);
7     long long Ans=0,PreSum=0,Max=0;
8     for(int i=1;i<=min(n,k);i++){
9         PreSum+=a[i];
10        Max=max(Max,b[i]);
11        Ans=max(Ans,PreSum+1ll*(k-i)*Max);
12    }
13    printf("%lld\n",Ans);
14 }
15
16 int main(){
17     scanf("%d%d",&c,&T);
18     while(T-->0) Solve();
19     return 0;
```


Contents

T1 抢修计划 (repair)

T2 风眼定位 (locate)

T3 环锁解除 (circle)

T4 风暴止息 (calm)

吐槽时间

数据点 1 20 Pts

- ▶ 当 $n \leq 20$ 时，可以询问整个区间得到全局第二大值的位置 s 。
- ▶ 再分别询问包含 s 的较小区间，逐个排除不可能成为最大值的位置。即使线性缩短区间，询问次数也不超过 20。
- ▶ 这一档提示我们把 s 当成比较基准。只要询问区间同时包含 s 与全局最大值，返回值就一定是 s 。

数据点 2 40 Pts

- ▶ 特殊性质 A 保证最大值位于端点。询问 $[1, n]$ 得到 s 后，最大值只可能在 1 或 n 。
- ▶ 若询问 $[1, s]$ 返回 s ，说明最大值在 s 左侧，所以答案是 1。否则最大值在右侧，答案是 n 。
- ▶ 只需两次询问，已经保留了正解中“用第二大值判断最大值所在方向”的核心步骤。与上一档的小范围算法合并，可以获得 40 分。

正解 100 Pts

- ▶ 首先询问 $[1, n]$ ，记返回位置为 s 。若 $s = 1$ ，最大值只能在右侧。若 $s = n$ ，最大值只能在左侧。
- ▶ 当 $1 < s < n$ 时询问 $[1, s]$ 。返回 s 当且仅当全局最大值也在这段区间内，因此可以确定最大值位于 s 的哪一侧。
- ▶ 下面只需在已经确定的一侧二分最大值位置。整个过程中，每次询问都让区间包含 s 。

正解 100 Pts

- ▶ 假设最大值在 s 左侧，维护答案属于 $[l, r] \subseteq [1, s-1]$ 。取上中点 m 并询问 $[m, s]$ 。
- ▶ 若返回 s ，说明最大值位于 $[m, s-1]$ ，令 $l = m$ 。否则最大值位于 $[l, m-1]$ ，令 $r = m-1$ 。
- ▶ 两次更新都保持答案位于 $[l, r]$ ，并严格缩短区间。右侧情形完全对称，使用下中点与询问 $[s, m]$ 。

正解 100 Pts

- ▶ 第一次询问得到 s ，至多再用一次询问判断方向。二分长度不超过 $n - 1$ ，至多使用 $\lceil \log_2(n - 1) \rceil$ 次询问。
- ▶ 因此总询问次数不超过 $2 + \lceil \log_2(n - 1) \rceil$ ，在 $n \leq 10^5$ 时远小于 40。
- ▶ 每次询问之外只进行常数运算，时间复杂度取决于交互库处理询问的方式，额外空间复杂度为 $\mathcal{O}(1)$ 。

正解 100 Pts

```
1
2 void Init(int c,int T){
3     return;
4 }
5
6 int Locate(int c,int n){
7     int Second=Query(1,n);
8     bool Left;
9     if(Second==1) Left=false;
10    else if(Second==n) Left=true;
11    else Left=(Query(1,Second)==Second);
12    if(Left){
13        int l=1,r=Second-1;
```

正解 100 Pts

```
1      while(l<r){
2          int Mid=(l+r+1)>>1;
3          if(Query(Mid,Second)==Second) l=Mid;
4          else r=Mid-1;
5      }
6      return l;
7  }
8  int l=Second+1,r=n;
9  while(l<r){
10     int Mid=(l+r)>>1;
11     if(Query(Second,Mid)==Second) r=Mid;
12     else l=Mid+1;
13 }
```


Contents

T1 抢修计划 (repair)

T2 风眼定位 (locate)

T3 环锁解除 (circle)

T4 风暴止息 (calm)

吐槽时间

数据点 1 ~ 5 20 Pts

- ▶ 当 $n \leq 10$ 时, 可以把当前控制位、已经解除的符号集合以及 x, y 是否交换作为状态, 使用最短路枚举四类操作。
- ▶ 免费解除操作只在当前符号等于下一个目标时发生。状态数为 $\mathcal{O}(n2^n)$, 能够完整验证旋转与交换的代价。

数据点 6 ~ 10 40 Pts

- ▶ 特殊性质中 $x = y = z$ 。交换不会改变旋转代价，额外进行交换只会增加代价，因此可以忽略交换操作。
- ▶ 每次只需比较从上一个目标位置顺时针与逆时针走到新目标位置时，经过多少个尚未解除的位置。取两种方向的较小值即可。
- ▶ 使用树状数组维护尚未解除的位置，单组时间复杂度为 $\mathcal{O}(n \log n)$ 。

数据点 11 ~ 19 76 Pts

- ▶ 当 $n \leq 10^3$ 时，可以用数组记录每个位置是否尚未解除。每次沿环分别向两个方向扫描，直接数出到达下一个目标前经过的未解除位置数量。
- ▶ 得到两个方向的距离后，仍然只需维护交换前后两个 DP 状态。每次转移耗时为 $\mathcal{O}(n)$ ，单组总时间复杂度为 $\mathcal{O}(n^2)$ 。
- ▶ 这一做法可以通过数据点 1 ~ 5 与 11 ~ 19，获得 56 分。再与特殊性质算法合并，可以获得 76 分。

正解 100 Pts

- ▶ 设 $d_{i,0}$ 表示已经按顺序解除前 i 个符号, 当前旋转代价仍为 (x, y) 时的最小代价。
- ▶ 设 $d_{i,1}$ 表示已经按顺序解除前 i 个符号, 当前旋转代价为 (y, x) 时的最小代价。
- ▶ 初始时还没有解除任何符号, 因此

$$d_{0,0} = 0, \quad d_{0,1} = z.$$

第二个状态表示在开始时先交换一次。

正解 100 Pts

- ▶ 从符号 b_{i-1} 的位置移动到 b_i 的位置。记顺时针经过的未解除位置数量为 l_i ，逆时针经过的数量为 r_i 。
- ▶ 已解除位置经过时不产生旋转代价，所以距离必须随着解除过程动态变化。把每个尚未解除的位置记为 1，在复制一遍的环上用树状数组查询区间和即可得到 l_i, r_i 。

正解 100 Pts

换向前的 DP 方程为

$$d_{i,0} = \min \begin{cases} d_{i-1,0} + l_i x, \\ d_{i-1,0} + r_i y, \\ d_{i-1,1} + l_i y + z, \\ d_{i-1,1} + r_i x + z. \end{cases}$$

前两项保持原方向，后两项从交换状态回到原方向。

正解 100 Pts

换向后的 DP 方程为

$$d_{i,1} = \min \begin{cases} d_{i-1,1} + l_i y, \\ d_{i-1,1} + r_i x, \\ d_{i-1,0} + l_i x + z, \\ d_{i-1,0} + r_i y + z. \end{cases}$$

每一项都唯一对应选择的旋转方向与是否交换，因此转移覆盖全部方案且不会引入非法方案。答案为 $\min(d_{n,0}, d_{n,1})$ 。

正解 100 Pts

- ▶ 每个符号只进行常数次数组查询，并在解除后把对应位置及复制位置删除。
- ▶ 单组时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。
- ▶ 多组数据之间需要重新建立树状数组，并覆盖本组使用的 DP 与位置数组。

正解 100 Pts

```
1 long long C[MAXN<<1],a[MAXN],b[MAXN],Dp[MAXN][2];
2 void Update(int Now,int Val){for(;Now<=2*n;Now+=lowbit(Now)) C[Now]+=1ll
   *Val;}
3 long long Query(int Now){long long Ret=0;for(;Now;Now-=lowbit(Now)) Ret
   +=C[Now];return Ret;}
4
5 long long GetD(int l,int r){
6     if(l>r) r+=n;
7     if(r-l>=n) r-=n;
8     return Query(r)-Query(l-1);
9 }
10
11 void Solve(){
12     scanf("%d%d%d%d",&n,&x,&y,&z);
13     for(int i=1;i<=2*n;i++) C[i]=lowbit(i);
14     for(int i=1;i<=n;i++) scanf("%lld",&a[i]),Pos[a[i]]=i;
15     for(int i=1;i<=n;i++) scanf("%lld",&b[i]);
16     Pos[0]=1;
17     Dp[0][0]=0;
```

正解 100 Pts

```
1  Dp[0][1]=z;
2  for(int i=1;i<=n;i++){
3      int dl=GetD(Pos[b[i-1]],Pos[b[i]])-GetD(Pos[b[i]],Pos[b[i]]);
4      int dr=GetD(Pos[b[i]]+1,Pos[b[i-1]]+n);
5      Dp[i][0]=min(min(Dp[i-1][0]+1ll*dl*x,Dp[i-1][0]+1ll*dr*y),min(Dp
6      [i-1][1]+1ll*dl*y+z,Dp[i-1][1]+1ll*dr*x+z));
7      Dp[i][1]=min(min(Dp[i-1][1]+1ll*dl*y,Dp[i-1][1]+1ll*dr*x),min(Dp
8      [i-1][0]+1ll*dl*x+z,Dp[i-1][0]+1ll*dr*y+z));
9      Update(Pos[b[i]],-1);
10     Update(Pos[b[i]]+n,-1);
11 }
12
13 int main(){
14     scanf("%d%d",&c,&T);
15     while(T-->0) Solve();
16     return 0;
17 }
```

Contents

T1 抢修计划 (repair)

T2 风眼定位 (locate)

T3 环锁解除 (circle)

T4 风暴止息 (calm)

吐槽时间

数据点 1 ~ 5 20 Pts

- ▶ 当 $n \leq 20$ 时，可以把所有定风楔状态压成一个二进制数。每个控制阵式对应一个固定异或掩码。
- ▶ 枚举全部状态并列出生成转移，解线性方程组得到期望。接管阈值内的状态直接使用最少操作数作为边界。
- ▶ 这一做法展示了状态只与“还需要多少个控制阵式”有关，具体编号可以进一步消去。

数据点 6 ~ 10 40 Pts

- ▶ 从编号 n 向 1 扫描。若当前第 i 枚定风楔失稳，只有启动编号为 i 的控制阵式才能改变它，并且后续更小编号的操作不会再次影响位置 i 。
- ▶ 因此此时必须启动 i ，并翻转 i 的全部正约数。这个贪心选择具有唯一性，得到的操作集合就是最少操作集合。
- ▶ 直接枚举约数可在 $\mathcal{O}(n\sqrt{n})$ 的时间内求出最少操作数。

数据点 11 ~ 15 60 Pts

- ▶ 设唯一最少操作集合的大小为 s 。随机启动一个仍在集合中的控制阵式，会把集合大小从 s 变成 $s - 1$ 。启动集合外的控制阵式，会把大小变成 $s + 1$ 。
- ▶ 于是随机过程只需要记录一个整数 s ，向下移动的概率为 s/n ，向上移动的概率为 $(n - s)/n$ 。
- ▶ 这一档可以按照状态数建立期望方程。完整范围需要把方程整理成一步递推。

数据点 16 ~ 19 76 Pts

- ▶ 特殊性质保证初始局面的最少操作数 $s \leq k$ 。CodeDay 会立即接管，因此随机阶段不会开始。
- ▶ 只需从大到小确定唯一的最少操作集合，并统计其中控制阵式的数量。答案为 $s \cdot n!$ 。
- ▶ 直接枚举约数的时间复杂度为 $\mathcal{O}(n\sqrt{n})$ 。将这一做法与前 15 个测试点的算法合并，可以获得 76 分。

正解 100 Pts

- ▶ 令 e_i 表示最少操作数从 i 第一次变成 $i-1$ 所需随机操作次数的期望。显然 $e_n = 1$ 。
- ▶ 当 $i < n$ 时，以 i/n 的概率一步到达 $i-1$ 。以 $(n-i)/n$ 的概率先到达 $i+1$ ，随后需要 e_{i+1} 回到 i ，还要重新等待 e_i 。
- ▶ 因此

$$e_i = 1 + \frac{n-i}{n}(e_{i+1} + e_i), \quad e_i = \frac{n + (n-i)e_{i+1}}{i}.$$

正解 100 Pts

- ▶ 若初始最少操作数 $s \leq k$, 答案就是 s 。若 $s > k$, 随机阶段需要依次从 s 降到 $s-1, \dots, k$, 期望为 $\sum_{i=k+1}^s e_i$ 。
- ▶ 接管后还要执行 k 次操作, 所以总期望为

$$k + \sum_{i=k+1}^s e_i.$$

- ▶ 除法在模 998,244,353 意义下乘逆元实现, 最后再乘 $n!$ 。

正解 100 Pts

- ▶ 最少操作集合可以从大到小枚举位置，并枚举当前编号的约数完成翻转。总枚举次数为 $\mathcal{O}(n \log n)$ 。
- ▶ 预处理逆元后，期望递推与求和均为 $\mathcal{O}(n)$ 。单组总时间复杂度为 $\mathcal{O}(n \log n)$ ，空间复杂度为 $\mathcal{O}(n)$ 。

正解 100 Pts

```
1 inline int Del(int a,int b){return a-b<0?a-b+Mod:a-b;}
2 inline int Mul(int a,int b){return 1ll*a*b%Mod;}
3 int c,T,n,k,cnt,Fact,Dp[MAXN],Ans;
4 int Cond[MAXN];
5 inline int Inv(int a){
6     int b=Mod-2,Ret=1;
7     while(b){
8         if(b&1) Ret=Mul(Ret,a);
9         a=Mul(a,a);
10        b>>=1;
11    }
12    return Ret;
13 }
14
15 void Solve(){
16     scanf("%d%d",&n,&k);
17     cnt=Ans=0;
```

正解 100 Pts

```
1 Fact=1;
2 for(int i=1;i<=n;i++) scanf("%d",&Cond[i]),Fact=Mul(Fact,i);
3 for(int i=n;i>=1;i--){
4     if(Cond[i]){
5         for(int j=1;j*j<=i;j++){
6             if(i%j==0){
7                 Cond[j]^=1;
8                 if(j*j!=i) Cond[i/j]^=1;
9             }
10        }
11        cnt++;
12        Cond[i]=0;
13    }
14 }
15 Dp[n]=1;
16 if(cnt<=k){
```

正解 100 Pts

```
1         printf("%d\n",Mul(cnt,Fact));
2         return;
3     }
4     for(int i=n-1;i>=1;i--) Dp[i]=Mul(Inv(i),Add(n,Mul(n-i,Dp[i+1])));
5     for(int i=cnt;i>=k+1;i--) Ans=Add(Ans,Dp[i]);
6     Ans=Mul(Add(Ans,k),Fact);
7     printf("%d\n",Ans);
8 }
9
10 int main(){
11     scanf("%d%d",&c,&T);
12     while(T-->0) Solve();
13     return 0;
14 }
```

Contents

T1 抢修计划 (repair)

T2 风眼定位 (locate)

T3 环锁解除 (circle)

T4 风暴止息 (calm)

吐槽时间